

Advanced Manual **Smart Contract Audit**

August 10, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

TeQoin

Tranche A, src/facets, re-audit

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	3	0	2	1
● Low-Risk	4	3	0	1
● Medium-Risk	2	1	0	1
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

TeQoin

Website	teqoin.io
Blockchain	Ethereum (L1) / TeQoin (L2)
Source Code	teqoin-mainnet-contracts, Tranche A, src/facets (re-audit of updated code)
Contract Address	Not deployed (undeployed mainnet candidate)

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Risk Classification	2
Audit Summary	3
Audit Scope	4
Audit Method	4
Automated Vulnerability Check	4
Manual Code Review	4
Used tools	4
Table of Contents	5
• Safety Checks That Can Never Actually Trigger	6
Code Snippet	6
Recommendation	6
• Monitoring Functions Loop Over Every Component	7
Code Snippet	7
Recommendation	7
• Full Transaction Data Is Copied Into The Event Log	8
Code Snippet	8
Recommendation	9
• The Escape Hatch Can Be Flooded Cheaply When Its Fee Is Set To Zero	10
Code Snippet	10
Recommendation	10
• Rotating To A New Operator Set Can Stall The Chain If The New Operators Are Offline	12
Code Snippet	12
Recommendation	12
• The Chain Trusts That Skipped Escape-Hatch Transactions Were Really Included	13
Code Snippet	13
Recommendation	14
• Flooding The Queue Can Postpone The Escape Hatch Deadline Guarantee	15
Code Snippet	15
Recommendation	16
• The Escape Hatch Can Be Disabled By A Broken Fee Destination	17
Code Snippet	17
Recommendation	17
• Pausing New Activity Does Not Stop Money Already In The Pipeline From Leaving	19
Code Snippet	19
Recommendation	19
Disclaimer	21

● Safety Checks That Can Never Actually Trigger

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
FAC-I1	Status: Acknowledged, kept deliberately. The checks guarding against the clock value growing too large to store are still present in these contracts. The original note asked the team to pick one consistent approach rather than to make a specific change, and they have done so: the genuinely misleading checks elsewhere in the project, such as the never-reached owner checks in the registry and the vaults, have been removed, while these harmless clock guards have been kept uniformly. That is a coherent position and we are satisfied with it.

Code Snippet

```
// still present, consistently, across the facets
if (block.timestamp > type(uint64).max || deadline > type(uint64).max)
    revert TimestampOverflow();
// kept as defensive insurance; unreachable in any realistic timeframe.
```

Recommendation

No action needed. The approach is now consistent across the project, which was the point of the original note.

● Monitoring Functions Loop Over Every Component

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
FAC-I2	Status: Acknowledged, unchanged. The read-only helper functions that summarise all installed modules for outside monitoring are unchanged in this update. No change was recommended, because these functions are only ever run by off-chain tools and never as part of a real transaction, so they cost users nothing and carry no security impact. The position stands.

Code Snippet

```
// DiamondLoupeFacet.sol : unchanged in this update
for (uint256 i; i < numFacets;) { ... } // view-only, off-chain use
```

Recommendation

No action needed. A note in the monitoring documentation stating the expected number of modules would close the point entirely, if the team wants to be thorough.

● Full Transaction Data Is Copied Into The Event Log

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
FAC-I3	Status: Resolved. The original concern was that the escape hatch both stored the full transaction and copied it into the event log, roughly doubling the cost of using a feature that exists as a safety valve. The redesign removes the duplication. The stored record now keeps only a fingerprint of the call along with its target, gas limit and length, and the full data appears once, in the event. That is the cheaper of the two arrangements and is the standard approach for this kind of feature. The consequence worth recording is that the event log is now the only place the call data exists. For this type of system that is normal and acceptable, because the log is permanently recoverable from the main chain, but it does mean log availability is load-bearing for anyone reconstructing the queue. This is the same theme raised in the libraries review about on-chain and off-chain agreement.

Code Snippet

```
// stored record: fingerprint only, no payload
fs.queue[index] = LibAppStorage.ForcedTransaction({
  queueId: queueId, callHash: committedCallHash,
  enqueuer: msg.sender, byteLength: byteLength,
  target: target, gasLimit: gasLimit, ...
}); // note: no data field

// the payload now appears once, in the event
emit ForcedCallEnqueued(..., data);
```

Recommendation

No further action needed. Make sure the software that reconstructs the queue reads the call data from event history and is tested against it, since there is no longer an on-chain copy to fall back on.

● The Escape Hatch Can Be Flooded Cheaply When Its Fee Is Set To Zero

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
FAC-L1	Status: Resolved. A minimum fee has been added to the shared settings and is now enforced whenever the fee is set, so the fee can no longer be configured down to zero. That closes the specific hole this finding described. Two honest caveats. The minimum chosen is very small, so in economic terms it is a guarantee that the fee is non-zero rather than a real deterrent; the actual cost of flooding the queue remains the network cost of each transaction, which is what does the work. And queue entries are still never removed, so the stored data still only grows. Neither changes the low rating, but see the separate finding in this report about how a flooded queue interacts with the new per-batch limit.

Code Snippet

```
// SystemConfig.sol : new floor, enforced on every change
uint256 public constant MIN_FORCED_INCLUSION_FEE = 1 gwei;

if (value < MIN_FORCED_INCLUSION_FEE || value > MAX_FORCED_INCLUSION_FEE) {
    revert ForcedInclusionFeeOutOfBounds(value);
}
```

Recommendation

No further action needed for the zero-fee case. If the team wants the fee to act as a genuine economic deterrent rather than a formality, the minimum would need to be

several orders of magnitude higher, balanced against keeping the safety valve affordable in a real emergency.

● Rotating To A New Operator Set Can Stall The Chain If The New Operators Are Offline

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
FAC-L2	Status: Still open, unchanged. The switch to a new operator set is still applied before the check that the caller is an approved operator, so the moment a scheduled switch becomes due, the outgoing operators can no longer submit and only the incoming ones can. If the incoming operators are not online and ready at that moment, no new batches can be produced until they are. Nothing is lost and the owner can schedule a corrected switch, but the chain can pause in the meantime. No code change was made here.

Code Snippet

```
// SequencerFacet.sol : unchanged ordering
_activateProposerSetIfReady(ss);           // swaps to the new set first
if (!ss.isProposer[msg.sender]) revert NotProposer(msg.sender);
// the outgoing operator can no longer submit the switch-over batch
```

Recommendation

Unchanged from the original review. Confirm each incoming operator is online and ready before scheduling a switch, and treat operator rotation as a monitored step in the operations checklist. A short overlap window where both sets may submit around the switch would remove the stall risk in code, if a code-level safeguard is wanted.

● The Chain Trusts That Skipped Escape-Hatch Transactions Were Really Included

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
FAC-L3	Status: Substantially strengthened, but the underlying assumption remains. This has improved a great deal. The queue accounting is now verified on the main chain rather than taken on trust: a batch must state where in the queue it starts, that value must match the real position, and the batch must commit to a fingerprint covering the exact set of queue items it claims to have handled, which the contract recomputes from its own stored records and compares. A proposer can therefore no longer advance the queue past items without committing to precisely which ones. The escape hatch itself was also redesigned so that each queue item produces one system transaction that must exist, rather than a raw transaction that could be waved through as invalid. What keeps this open is the part that was always the real dependency: whether those committed transactions were actually carried out on the other chain is still established by the fraud-detection system, which remains incomplete in this phase. The accounting hole is closed; the execution guarantee still rests on the trusted path.

Code Snippet

```
// SequencerFacet.sol : queue accounting now verified on-chain
if (header.forcedQueueStart != head)
    revert InvalidForcedQueueStart(head, header.forcedQueueStart);

// the committed set is recomputed from stored records and compared
```

```
actualRoot = LibForcedTransaction.computeRangeRoot(head, callHashes);  
if (actualRoot != header.forcedTransactionsRoot)  
    revert ForcedTransactionsRootMismatch(...);
```

Recommendation

Keep the trust assumption stated plainly in the public documentation for this phase, and prioritise completing the fraud-detection system, which is the real resolution and is already the project's stated direction. The on-chain commitment added here is exactly the right groundwork for that.

● Flooding The Queue Can Postpone The Escape Hatch Deadline Guarantee

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
FAC-L4	Status: New in this review, arising from the per-batch limit. The escape hatch promises that a queued item cannot be left waiting past its deadline, and the contract enforces this by refusing a batch that leaves an overdue item behind. The update adds a sensible cap of 256 items handled per batch, and to avoid that cap making batches impossible, the overdue check is skipped whenever a batch handles a full 256. The side effect is that the deadline guarantee can be postponed by filling the queue. Someone who adds a large number of entries ahead of a victim's entry forces the queue to be worked through 256 at a time, and for every one of those full batches the overdue protection does not apply. Progress is still guaranteed and the queue does drain, so this delays rather than permanently blocks, and it costs the attacker the network fee for every entry they add, which is real money at scale. But it does mean the deadline is a soft guarantee under load rather than a hard one, and the cost scales predictably with how long an attacker wants to delay someone.

Code Snippet

```
// SequencerFacet.sol : overdue check suppressed on a full chunk
if (count < MAX_FORCED_TRANSACTIONS_PER_BATCH
    && block.timestamp >= next.inclusionDeadline) {
    revert OverdueForcedTransaction(consumed, next.inclusionDeadline);
}
// with MAX_FORCED_TRANSACTIONS_PER_BATCH = 256, an attacker who adds
// N entries ahead of a victim suppresses the check for N/256 batches.
```

Recommendation

Decide deliberately how strong the deadline promise should be. Options that preserve it include requiring that a full batch still makes progress on the oldest overdue item specifically, or tracking a separate deadline for the queue head that a full chunk cannot suppress. Raising the minimum fee, as discussed in the fee finding above, also directly raises the cost of this. At minimum, monitor queue depth and alert when it grows far beyond normal, since a flood is visible well before it causes harm.

● The Escape Hatch Can Be Disabled By A Broken Fee Destination

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
FAC-M1	Status: Resolved. This is fixed exactly as recommended, and it matters because the feature at stake is the one that guarantees users can get their transactions through even if the operator ignores them. Previously the fee was pushed to the collection address as part of every submission, so an address that refused payments would have made the whole escape hatch unusable. Fees are now simply counted and held inside the contract when someone uses the escape hatch, and moving them to the collection address is a separate action that anyone can trigger. A broken destination can now only delay fee collection; it can no longer stop anyone from using the safety valve. The code says as much in its own comment, and a matching view was added so the accrued balance is visible.

Code Snippet

```
// ForcedInclusionFacet.sol : fees accrue, no push during submission
fs.accruedFees += expectedFee;

// collection is a separate, permissionless action
function flushForcedInclusionFees() external returns (uint256 amount) {
    // 'A broken destination can delay fee collection but cannot
    //   censor calls.'
}
```

Recommendation

No further action needed. Include the accrued balance in routine monitoring so fees are actually collected rather than quietly accumulating.

● Pausing New Activity Does Not Stop Money Already In The Pipeline From Leaving

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
FAC-M2	Status: Still open, unchanged. The two emergency switches are still independent. Stopping new batches from being submitted does not stop a batch that was already submitted from completing its waiting period and being marked final, because that step has no pause check of its own, and the actual payout is controlled by the separate withdrawals switch. An operator responding to a suspected bad batch therefore still has to remember to flip both, and flipping only the obvious one leaves funds exposed. One related improvement is worth noting even though it does not close this finding: withdrawals can now be redirected by their recipient before payout, which gives responders one more tool in a stuck-funds situation. The two-switch coordination problem itself is unchanged.

Code Snippet

```
// SequencerFacet.sol : submission is gated
LibProtocol.enforceNotPaused(LibProtocol.PAUSE_BATCH_SUBMISSION);

// but finalizeBatch(...) still has no pause check, and payout is
// gated only by the separate withdrawals switch in BridgeFacet:
LibProtocol.enforceNotPaused(LibProtocol.PAUSE_WITHDRAWALS);
```

Recommendation

Unchanged from the original review. Treat the two switches as a single action in the emergency playbook: when stopping submissions because a batch is suspect, always pause withdrawals at the same time. For a stronger safeguard, consider a single

emergency stop that halts both at once, so money cannot keep moving while responders decide what to do.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

Advanced Manual **Smart Contract Audit**

August 10, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

TeQoin

Tranche A, src/escrow, re-audit

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	4	2	1	1
● Low-Risk	4	0	0	4
● Medium-Risk	3	0	0	3
● Critical-Risk	1	0	0	1

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

TeQoin

Website	teqoin.io
Blockchain	Ethereum (L1) / TeQoin (L2)
Source Code	teqoin-mainnet-contracts, Tranche A, src/escrow (re-audit of updated code)
Contract Address	Not deployed (undeployed mainnet candidate)

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Risk Classification	2
Audit Summary	3
Audit Scope	4
Audit Method	4
Automated Vulnerability Check	4
Manual Code Review	4
Used tools	4
Table of Contents	5
• An Unused Error Message And A Check That Can Never Trigger	6
Code Snippet	6
Recommendation	6
• The Two Vaults Contain Duplicate Copies Of The Same Safety Logic	7
Code Snippet	7
Recommendation	7
• The Withdrawal Speed Limit Is Sized From The Peak Amount Held, Not The Current Amount	9
Code Snippet	9
Recommendation	9
• Very Small Limits Combined With Frequent Withdrawals May Never Replenish	10
Code Snippet	10
Recommendation	11
• The Vault's Pause Button Stops Money Leaving But Not Money Arriving	12
Code Snippet	12
Recommendation	12
• Some Limiter Resets Happen Silently, Especially For The Token Vault	13
Code Snippet	13
Recommendation	13
• Setting A Token's Limit To Zero Loosens It Instead Of Stopping It	14
Code Snippet	14
Recommendation	14
• If The Shared Settings Cannot Be Read, The Limit Is Removed With No Visible Signal	15
Code Snippet	15
Recommendation	15
• A Perfectly Valid Setting Can Silently Switch The Speed Limit Off	16
Code Snippet	16
Recommendation	17
• Twice The Intended Amount Can Leave Around A Window Boundary	18

Code Snippet	18
Recommendation	18
• A Blocked Recipient Can Strand A Withdrawal, Despite A Note Suggesting Otherwise	19
Code Snippet	19
Recommendation	20
• Tokens That Take A Cut On Transfer Can Be Deposited But Never Withdrawn, Trapping The Funds	21
Code Snippet	21
Recommendation	22
Disclaimer	23

● An Unused Error Message And A Check That Can Never Trigger

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
ESC-I1	Status: Resolved. The unused error message for rejecting direct transfers has been deleted, so nobody will go looking for a handler that was never there. The never-reached owner check has also been removed from both vaults and replaced with a comment explaining where that validation actually happens. Both vaults still check the addresses that genuinely are unvalidated elsewhere.

Code Snippet

```
// both vaults, updated
// ContractOwner2Step validates owner_ before this constructor body runs.
if (bridge_ == address(0) || systemConfig_ == address(0)) revert ZeroAddress();

// error DirectTransfersRejected();    <- deleted
```

Recommendation

No further action needed.

● The Two Vaults Contain Duplicate Copies Of The Same Safety Logic

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
ESC-I2	Status: Substantially improved, but still partly open. The most valuable part of this has been done. The rate-limit arithmetic, which is the fiddly and error-prone part, has been pulled out into a single shared piece of code that both vaults now use. The specific drift this finding warned about, where one vault recorded an event and the other did not, has also been fixed. That materially reduces the risk. What keeps it open is that the same update introduced fresh duplication. The new monitoring status view and the new window-length validation were each written twice, once in each vault, in near-identical form. So the pattern that caused the original drift is still in play for the newly added code.

Code Snippet

```
// now shared (good):
LibEscrowRateLimit.capacity / currentUsage / available / ceilDiv

// still written twice, once per vault:
rateLimitStatus(...) // near-identical in both
_validateWindowLength(...) // near-identical in both
pause / unpause / _guardian / _timestamp
```

Recommendation

Extend what was already started: move the status view and the window-length validation into the same shared library as the arithmetic, so the ETH vault becomes the single-key case of the token vault. If some duplication is kept deliberately, add an

automated check that flags any unintended difference between the two files, so a future fix cannot land in only one of them.

● The Withdrawal Speed Limit Is Sized From The Peak Amount Held, Not The Current Amount

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
ESC-I3	Status: Acknowledged, unchanged by design. This behaviour is the same in the updated code, which is correct. Sizing the limit from the highest amount held rather than the current amount is what guarantees everyone can still exit in full within the allowed time, and a limit based on a shrinking balance could not offer that. No change was recommended and none was made. The only consequence to keep in mind is that the limit looks loosest, in relative terms, right after a large amount has already been withdrawn.

Code Snippet

```
// unchanged in both vaults
if (_totalLocked > drainBasis) drainBasis = _totalLocked; // peak, not live
// cleared only when the vault empties completely.
```

Recommendation

No code change needed. Keep the note in the operations guide so anyone reading the remaining allowance during an incident understands why it can look large compared with the balance on hand.

● Very Small Limits Combined With Frequent Withdrawals May Never Replenish

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
ESC-I4	Status: New in this review, introduced by the improved limiter. The new limiter replenishes gradually based on the time since the last withdrawal, and it moves that last-withdrawal marker forward every time a withdrawal happens. The amount replenished is worked out by whole units only, so any fraction smaller than one unit is discarded rather than carried forward. If the limit is very small compared with the length of the window, a stream of frequent small withdrawals can therefore replenish nothing at all each time, and the discarded time is lost permanently, so the allowance may never recover. This errs on the strict side rather than the permissive one, so it is not a security weakness: the worst outcome is that withdrawals are refused sooner than intended. It is also unlikely to be reached in practice, because the limit is floored by the amount held and would have to be smaller than the number of seconds in the window, which for any realistic token balance it will not be. Noting it so the behaviour is understood rather than discovered.

Code Snippet

```
// LibEscrowRateLimit.sol : replenishment is whole units only
uint256 replenished = Math.mulDiv(capacity_, elapsed, windowLength);
// if capacity_ * elapsed < windowLength this is 0

// and the marker still moves forward, discarding that elapsed time:
w.used = used + amount;
w.start = now_;
```

Recommendation

No urgent change needed. If the team wants to close it completely, either do not move the last-withdrawal marker forward when nothing was replenished, so the elapsed time keeps accumulating, or require any configured limit to be at least as large as the number of seconds in the window. Either is a small change and removes the edge case entirely.

● The Vault's Pause Button Stops Money Leaving But Not Money Arriving

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
ESC-L1	Status: Resolved. Both vaults now check the pause flag on the way in as well as on the way out, so a single pause stops the vault completely rather than only stopping payouts. The documentation has been updated to say so. This removes the situation where users could keep depositing into a vault that was known to be unable to pay out, and it means an emergency responder no longer has to remember a second action on a different contract just to stop deposits into a paused vault.

Code Snippet

```
// both vaults, updated
function depositETH() external payable override onlyBridge {
    if (paused) revert EscrowPaused(); // new

function depositERC20(...) external override onlyBridge nonReentrant ... {
    if (paused) revert EscrowPaused(); // new

/// @notice Halts both deposits and releases.
```

Recommendation

No further action needed. Keep the wider system pause in the emergency playbook as well, since stopping deposits at the bridge entrance remains a separate control.

● Some Limiter Resets Happen Silently, Especially For The Token Vault

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
ESC-L2	Status: Resolved. Both gaps are closed. The ETH vault now records an event when the limiter resets because the vault emptied, which it previously did silently. The token vault, which had no reset event at all, now declares one and records it per token, both when a token empties and when the limiter is found to be unlimited. Outside monitoring can now reconstruct limiter state for both vaults from the event history.

Code Snippet

```
// ETH vault: the empty-out path now records the reset
windowStart = _timestamp();
emit RateLimitWindowReset(windowStart);          // new

// token vault: a per-token event now exists
event RateLimitWindowReset(address indexed token, uint64 windowStart);
emit RateLimitWindowReset(token, _timestamp());
```

Recommendation

No further action needed.

● Setting A Token's Limit To Zero Loosens It Instead Of Stopping It

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
ESC-L3	Status: Resolved. Setting a token's limit to zero used to be silently reinterpreted as use the default, which meant an operator trying to tighten a token during an incident could accidentally loosen it. The fix is exactly the one recommended. A zero value is now refused outright with its own clear error, and a separate, clearly named action has been added for putting a token back onto the default. A separate marker now records whether a token has an explicit limit, so zero is no longer used to mean anything, and clearing a limit records its own event.

Code Snippet

```
// token vault, updated
if (capacity_ == 0) revert ZeroTokenCapacity(token);
// no longer silently reinterpreted as the default
hasTokenCapacity[token] = true;

function clearTokenRateLimit(address token) external onlyOwner { ... }
emit TokenRateLimitCleared(token);
```

Recommendation

No further action needed. Remember that a genuine zero limit is still not achievable, because the guaranteed-exit floor overrides the configured value, so the pause button remains the correct tool for fully stopping a single token.

● If The Shared Settings Cannot Be Read, The Limit Is Removed With No Visible Signal

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
ESC-L4	Status: Resolved. Both vaults still remove the limit if the shared settings cannot be read, which remains the right call, because a broken settings contract must not be able to trap users. What was missing was any way to tell that apart from a deliberate no-limit setting. A monitoring view has now been added that reports exactly which of four situations applies: the limit is active, it is deliberately unlimited, the settings could not be read, or the window is longer than the challenge period. It also returns the challenge period, the current limit, and the amount still available, so monitoring has everything it needs in one call.

Code Snippet

```
// both vaults, new monitoring view
enum RateLimitStatus {
    Active,
    IntentionallyUnlimited,
    ConfigUnavailable,           // the degraded case, now visible
    WindowExceedsChallengePeriod
}
function rateLimitStatus() external view returns (
    RateLimitStatus status, uint64 challengePeriod_,
    uint256 capacity_, uint256 available_);
```

Recommendation

No further action needed in the code. Wire this view into the monitoring and alert on anything other than an active limit, so the degraded state is actually noticed rather than merely being visible.

● A Perfectly Valid Setting Can Silently Switch The Speed Limit Off

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
ESC-M1	Status: Resolved. Both halves of this are addressed. The settings can no longer be put into the disabling state from the vault side: a new validation step rejects any window longer than the current challenge period, and it is applied both when the vault is created and whenever the limit is changed. If the challenge period cannot be read at that moment, the change is refused rather than allowed through, which is the safe direction for a configuration change. Separately, the new monitoring view reports the window-exceeds-challenge-period case explicitly, so it can no longer hide. One residual remains, and it is now a process matter rather than a code gap. The challenge period can still be lowered elsewhere in governance, which could push an existing vault into the disabling state without any transaction touching the vault. The difference is that this is now immediately visible through the monitoring view instead of being silent.

Code Snippet

```
// both vaults, new validation on create and on change
function _validateWindowLength(uint64 candidate) private view {
    try systemConfig.challengePeriod() returns (uint64 period) {
        if (period == 0 || candidate > period)
            revert WindowExceedsChallengePeriod(candidate, period);
    } catch {
        revert ChallengePeriodUnavailable();
    }
}
```

Recommendation

No further code change needed. Keep the process half: treat any reduction of the challenge period as requiring a matching review of both vaults, and alert on the monitoring view so that if the two ever do drift out of step it is caught immediately rather than at the next incident.

● Twice The Intended Amount Can Leave Around A Window Boundary

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
ESC-M2	Status: Resolved. The limiter no longer works in fixed blocks of time that reset all at once. Allowance now replenishes gradually and continuously as time passes, so there is no predictable moment at which the counter drops back to zero. Withdrawing the full allowance and then immediately withdrawing it again is now refused, and recovering the full allowance takes a full window rather than happening instantly at a boundary. The documentation has been corrected to describe the actual behaviour, and the shared library explains the reasoning. The reaction time the limiter is meant to buy is now the real figure rather than half of it.

Code Snippet

```
// LibEscrowRateLimit.sol : gradual replenishment, no reset boundary
uint256 elapsed = currentTimestamp - lastUpdatedAt;
if (elapsed >= windowLength) return 0;
uint256 replenished = Math.mulDiv(capacity_, elapsed, windowLength);
return replenished >= recordedUsage ? 0 : recordedUsage - replenished;

// header, corrected:
// 'capacity does not reset at a predictable boundary where two
// full withdrawals could execute back-to-back.'
```

Recommendation

No further action needed. Note the related edge case recorded separately in this report about very small limits, which comes from the same replenishment arithmetic.

● A Blocked Recipient Can Strand A Withdrawal, Despite A Note Suggesting Otherwise

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
ESC-M3	Status: Resolved. This has been fixed properly rather than by only correcting the misleading note. A real recovery route now exists: a withdrawal that has been proven but not yet paid out can be pointed at a different destination address, after which the payout goes to the new address instead. The amount and the underlying proof are untouched, so nothing about the withdrawal's validity changes. The permission design here is better than what we suggested. We proposed letting the original sender redirect, but the team correctly identified that this would let a sender take back a payment they had already made. Instead, only the destination address named in the proof can authorize a replacement, which closes the recovery gap without creating a way to revoke a completed transfer. The note in the vault has also been rewritten and is now honest about the one case that remains: a destination contract with no way to authorize anything can still end up stuck.

Code Snippet

```
// BridgeFacet.sol : new recovery route
function redirectProvenWithdrawal(
    LibWithdrawal.WithdrawalMessage calldata message,
    address newRecipient
) external returns (bytes32 withdrawalId) {
    // only the address named in the proof may authorize:
    if (msg.sender != message.to)
        revert NotWithdrawalRecipient(msg.sender, message.to);
    // must be proven and not yet paid out
    record.recipientOverride = newRecipient;
}
```

Recommendation

No further action needed. Keep the front-end warning about choosing a destination carefully, since the remaining stuck case is a destination that cannot act at all, and make sure the redirect route is documented somewhere users in that situation will actually find it.

● Tokens That Take A Cut On Transfer Can Be Deposited But Never Withdrawn, Trapping The Funds

Critical-Risk - Will definitely cause problems, this needs to be adjusted

Error Code	Description
ESC-C1	Status: Resolved. This was the most serious finding in the original escrow review. A token that quietly takes a cut on every transfer could be deposited and credited on the other chain, but could never be withdrawn, because every withdrawal demands that the exact amount move in and out. The funds were accepted into a place they could never leave, and the problem stayed hidden until the first person tried to withdraw. The fix is exactly the one recommended and it is the cheapest possible one. The vault already knew, at the moment of deposit, that less had arrived than was requested; it now simply refuses the deposit in that case. A silent permanent loss has become a clean, immediate rejection on the very first deposit, before anything is credited on the other chain and before any user is exposed. The documentation has been rewritten to match, and now states plainly that these tokens are not supported and are rejected on the way in.

Code Snippet

```
// L1ERC20Escrow.sol : updated
received = afterBal - before;
if (received == 0) revert NothingReceived(token);
if (received != amount)
    revert UnexpectedDepositAmount(token, amount, received); // new

// header, rewritten:
// 'FEE-ON-TRANSFER: NOT supported. ... the deposit reverts unless the
// exact requested amount arrived. This prevents L2 credit from being
// created for a token that cannot later satisfy a withdrawal.'
```

Recommendation

No further action needed to clear this finding. The one case it cannot cover still stands and belongs in the token-approval checks rather than the vault: a token whose cut is currently switched off but can be switched on later will pass the deposit check today and could strand funds afterwards. Keep that on the review criteria alongside the changing-details case from the registry review.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

Advanced Manual **Smart Contract Audit**

August 10, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

TeQoin

Tranche A, src/bridge, re-audit

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	4	1	0	3
● Low-Risk	4	1	0	3
● Medium-Risk	2	0	0	2
● Critical-Risk	1	0	0	1

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

TeQoin

Website	teqoin.io
Blockchain	Ethereum (L1) / TeQoin (L2)
Source Code	teqoin-mainnet-contracts, Tranche A, src/bridge/TokenPairRegistry.sol (re-audit of updated code)
Contract Address	Not deployed (undeployed mainnet candidate)

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Risk Classification	2
Audit Summary	3
Audit Scope	4
Audit Method	4
Automated Vulnerability Check	4
Manual Code Review	4
Used tools	4
Table of Contents	5
• Safety Checks That Can Never Actually Trigger	6
Code Snippet	6
Recommendation	6
• One Lookup Gives An Unclear Error When Asked For Something Out Of Range	7
Code Snippet	7
Recommendation	7
• Two Small Tidiness Points	8
Code Snippet	8
Recommendation	8
• Adding A New Status Shifted The Numbering Of An Existing One	9
Code Snippet	9
Recommendation	9
• Anyone Can Add A Token, And The List Only Ever Grows	10
Code Snippet	10
Recommendation	10
• A Hand-Written Check On Token Names Misses One Case	12
Code Snippet	12
Recommendation	12
• The Budget For Reading Token Details Is A Fixed Number That May Be Too Tight	13
Code Snippet	13
Recommendation	13
• A Token's Real Name Only Survives In The Event History	14
Code Snippet	14
Recommendation	14
• A Key Address Is Left Out Of The Deployment Safety Check, And A Wrong Value	15
Traps Deposits	15
Code Snippet	15
Recommendation	15
• Blocking A Token Is Permanent, With No Way Back	16

Code Snippet	16
Recommendation	16
• If A Token's Details Change, It Can Be Permanently Locked Out Of The Bridge	17
Code Snippet	17
Recommendation	18
Disclaimer	19

● Safety Checks That Can Never Actually Trigger

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
TPR-I1	Status: Resolved. Originally the setup code re-checked that the owner address was not blank, even though the building block it is based on already rejects that before the check is reached. That check has been removed and replaced with a comment explaining where the validation actually happens, which was the recommendation. A similar check against a network identifier of zero remains, but it is harmless defensive code and keeping it was offered as an acceptable option.

Code Snippet

```
// TokenPairRegistry.sol : updated
// ContractOwner2Step validates owner_ before this constructor body runs.
if (l2TokenFactory_ == address(0) || l2Bridge_ == address(0)) {
    revert ZeroAddress();
}
// the unreachable owner_ check has been dropped.
```

Recommendation

No further action needed. If the team later standardises on removing all unreachable checks across the project, the remaining network-identifier check here would be the last one in this file.

● One Lookup Gives An Unclear Error When Asked For Something Out Of Range

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
TPR-I2	Status: Resolved. Asking the token list for an item beyond its end used to fail with a generic system error instead of a clear named one. The lookup now checks the range first and reports a specific out-of-bounds error naming both the requested position and the actual size. The team also added the paged lookup that was suggested alongside it, capped at a sensible maximum page size, which makes reading the list much easier for outside tools.

Code Snippet

```
// TokenPairRegistry.sol : updated
uint256 count = _registeredTokens.length;
if (index >= count) revert IndexOutOfBounds(index, count);

// new paged lookup, bounded:
function registeredTokens(uint256 offset, uint256 limit) ...
if (limit > MAX_PAGE_SIZE) revert PageTooLarge(limit, MAX_PAGE_SIZE);
```

Recommendation

No further action needed. Both the clear error and the paged lookup are in place and covered by tests.

● Two Small Tidiness Points

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
TPR-I3	Status: Resolved. Both points have been addressed. The internal references used to read a token's details are now taken from a checked source rather than built from plain text, so a typo would be caught automatically instead of silently reading the wrong function. A blocked-at timestamp has also been added to each token's record and is filled in when a token is blocked, so the full history can now be read from the record itself rather than reconstructed from event history. As expected, the new field fits in existing space and costs no extra storage.

Code Snippet

```
// TokenPairRegistry.sol : updated
name    = _readString(token, IERC20Metadata.name.selector);
symbol  = _readString(token, IERC20Metadata.symbol.selector);

// record now keeps a blocked-at time:
uint64 deniedAt;
pair.deniedAt = _timestamp();    // set inside denyToken
```

Recommendation

No further action needed.

● Adding A New Status Shifted The Numbering Of An Existing One

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
TPR-I4	Status: New in this review. The new reversible suspend feature added a Suspended status, and it was inserted in the middle of the existing list rather than at the end. As a result the numeric value that represents Blocked has shifted from 3 to 4. On the chain itself this is completely safe, because nothing has been deployed yet and there is no stored data to migrate. The point worth flagging is off-chain: the token's status is returned as a number by the record lookup, so any monitoring tool, indexer, or dashboard that maps those numbers to names must be updated, or it will silently report a blocked token as something else. This is the same category of on-chain and off-chain agreement raised in the libraries review.

Code Snippet

```
// before:  None=0, Registered=1, Enabled=2, Denied=3
// after:   None=0, Registered=1, Enabled=2, Suspended=3, Denied=4
//
// pairOf() returns this value to outside tools, so anything
// mapping the number to a name must be updated to match.
```

Recommendation

Check every off-chain consumer that reads a token's status number and update it to the new list before launch. For any future additions, adding a new status at the end of the list avoids shifting existing values. Since nothing is deployed yet, this costs nothing to get right now.

● Anyone Can Add A Token, And The List Only Ever Grows

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
TPR-L1	Status: Still open. Adding a token to the registry is still open to anyone and costs only the basic network fee, the list is still never cleaned up, and the event is still named in a way that reads like an approval. None of the three suggested options was taken. The updated code does make the tooling side better, because the new paged lookup means outside tools no longer need one call per entry, but that addresses convenience rather than the concern itself. One small point in the other direction: each registration now also stores the token's name and symbol text, so a spam entry occupies a little more permanent space than before. The spammer still pays for it, so this remains self-limiting, and the risk stays low.

Code Snippet

```
// TokenPairRegistry.sol : unchanged access control
function registerToken(address l1Token) external returns (address l2Token) {
    // still no access control, still no cost beyond gas
    _registeredTokens.push(l1Token); // still never pruned
    _names[l1Token] = name; // new: more stored per entry
    _symbols[l1Token] = symbol;
    emit PairRegistered(...); // still reads like approval
}
```

Recommendation

Unchanged from the original review. Open discovery is a reasonable choice and does not need to be removed, but the wording should be made unambiguous so a listing can never be mistaken for approval. Consider only showing tokens in the public list once they have actually been switched on, or adding a small refundable deposit to price out bulk spam. Now that a token can be suspended and resumed, making registration

owner-only is also a lighter change than it was before, since the recovery paths that used to depend on open registration are no longer needed.

● A Hand-Written Check On Token Names Misses One Case

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
TPR-L2	Status: Resolved. The hand-written sanity check on a token's name used to miss one case, letting a badly-formed name slip through and fail a moment later with an unclear system error. The missing check has been added exactly as recommended: the code now also confirms the data is long enough to hold the full padded text before decoding it. Every badly-formed token now fails with the contract's own clear message, and the decoding step that follows can no longer fail.

Code Snippet

```
// TokenPairRegistry.sol : updated
if (offset != 32 || length > data.length - 64)
    revert MetadataUnavailable(token);
uint256 paddedLength = (length + 31) & ~uint256(31);
if (data.length < 64 + paddedLength) revert MetadataUnavailable(token);
value = abi.decode(data, (string)); // now provably safe
```

Recommendation

No further action needed. Adding a test that feeds in a deliberately badly-formed name would lock the behaviour in, if one is not already present.

● The Budget For Reading Token Details Is A Fixed Number That May Be Too Tight

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
TPR-L3	Status: Resolved. The budget for reading a token's details was a single fixed number that could not be changed and risked rejecting legitimate but more complex tokens. It is now an adjustable setting, and the default has been doubled, giving much more headroom. Changes are limited to a safe range and are announced by an event, so the protection against a hostile token is kept while the project can now adapt to future tokens or network pricing changes without redeploying. The separate limit on response size, which is the part that actually guards against abuse, has been kept unchanged.

Code Snippet

```
// TokenPairRegistry.sol : updated
uint256 public constant MIN_METADATA_CALL_GAS = 30_000;
uint256 public constant MAX_METADATA_CALL_GAS = 500_000;
uint256 public constant DEFAULT_METADATA_CALL_GAS = 100_000;
// (was a fixed, unchangeable 50_000)

function setMetadataCallGas(uint256 newLimit) external onlyOwner { ... }
// bounded, owner-only, and emits MetadataCallGasUpdated
```

Recommendation

No further action needed. Worth remembering that the maximum setting allows a hostile token to consume a larger share of the caller's budget than before, so keep the setting near the default unless a specific token needs more.

● A Token's Real Name Only Survives In The Event History

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
TPR-L4	Status: Resolved. The registry used to store only fingerprints of a token's name and symbol, which meant the actual text needed to complete bridging on the other chain existed nowhere except the public event history. The actual text is now stored directly in the contract and can be read back through a new lookup function, exactly as recommended. This removes a fragile reconstruction step from the path that moves value, and the text is kept up to date when a registration is refreshed.

Code Snippet

```
// TokenPairRegistry.sol : updated
mapping(address => string) private _names;
mapping(address => string) private _symbols;

function metadataOf(address l1Token) external view
    returns (string memory name, string memory symbol, uint8 decimals);
```

Recommendation

No further action needed. Point the off-chain deposit processor at this lookup rather than at event history, so the improvement is actually used in practice.

● A Key Address Is Left Out Of The Deployment Safety Check, And A Wrong Value Traps Deposits

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
TPR-M1	Status: Resolved. The deployment safety check used to verify most of the key addresses but left out the factory that creates the matching token on the other chain, which is the address that determines where every bridged token will live. A wrong value would have passed every check, and users would only have found out when their deposits could not be completed and their funds were stuck. The missing address has now been added at every level: it is exposed by the registry, it is a required setup value, it is rejected if blank, and it is cross-checked against the registry during deployment, in the same way as the neighbouring addresses.

Code Snippet

```
// DiamondInit.sol : updated
function L2_TOKEN_FACTORY() external view returns (address); // now exposed
address l2TokenFactory; // now a setup value
if (a.l2TokenFactory == address(0)) revert ZeroAddress("l2TokenFactory");

_requireAddress("tokenPairRegistry.l2TokenFactory",
    a.l2TokenFactory, registry.L2_TOKEN_FACTORY()); // now cross-checked
```

Recommendation

No further action needed for this check. The wider question raised alongside it still stands and is tracked in the escrow and L2 reviews: there is still no way to cancel or refund a deposit that cannot be completed on the other chain, so any other cause of the same outcome would still leave funds stuck.

● Blocking A Token Is Permanent, With No Way Back

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
TPR-M2	Status: Resolved. Blocking a token used to be the only way to stop it, and it could never be undone, so a precautionary stop during an incident or a mistake against the wrong token was permanent. A reversible pause has now been added exactly as recommended: a token that is live can be suspended and later resumed, and suspending correctly stops new deposits while leaving existing withdrawals able to settle. Permanent blocking is kept for the genuinely final case, which is the right split. The behaviour is covered by a test that walks through suspending, resuming, and then blocking, and confirms that a blocked token cannot be resumed.

Code Snippet

```
// TokenPairRegistry.sol : new reversible pause
function suspendToken(address l1Token) external onlyOwner;
//   Enabled -> Suspended
function resumeToken(address l1Token) external onlyOwner;
//   Suspended -> Enabled

// suspending stops new deposits:
isBridgeable(...) == (status == PairStatus.Enabled) // false while Suspended
// withdrawals still settle, because l2TokenOf ignores status.
```

Recommendation

No further action needed. Make sure the governance runbook distinguishes the two actions clearly, so that a responder reaching for a temporary stop uses suspend and not the permanent block.

● If A Token's Details Change, It Can Be Permanently Locked Out Of The Bridge

Critical-Risk - Will definitely cause problems, this needs to be adjusted

Error Code	Description
TPR-C1	Status: Resolved. This was the most serious finding in the original review. A token's details were locked in the moment anyone added it, and if those details later changed the token could never be switched on, never be added again, and never be bridged. Because adding a token is open to anyone, an outsider could trigger this deliberately on a major token. The fix is exactly the one recommended. An owner-only refresh function has been added that re-reads a token's current details and updates its record, and it is deliberately limited to tokens that have not yet been switched on. That limit is the important part, because it means a token already in live use can still never be quietly repointed to a different address, so no user funds are put at risk by the new ability. A test now walks the whole path: add a token, change its details, refresh, switch it on successfully, and then confirm that refreshing is refused once it is live.

Code Snippet

```
// TokenPairRegistry.sol : new recovery path
function refreshRegisteredToken(address l1Token)
    external onlyOwner returns (address l2Token)
{
    Pair storage pair = _pairs[l1Token];
    if (pair.status != PairStatus.Registered) // pre-activation only
        revert InvalidPairStatus(l1Token, pair.status);
    // re-reads current details and rewrites the record
}
// a token that has ever been Enabled can never be repointed.
```

Recommendation

No further action needed to clear this finding. One small residual is worth handling in the operating procedure rather than the code: refreshing and switching on are two separate steps, so someone able to change a token's details could in principle change them again in between and cause the second step to fail. That is now only a nuisance rather than a permanent lock-out, since the refresh can simply be repeated. Scheduling the refresh and the switch-on together as a single batched governance action would remove even that gap.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

Advanced Manual

Smart Contract Audit

August 10, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

TeQoin

Tranche A, src/libraries, re-audit

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	4	1	2	1
● Low-Risk	3	2	1	0
● Medium-Risk	1	1	0	0
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

TeQoin

Website	teqoin.io
Blockchain	Ethereum (L1) / TeQoin (L2)
Source Code	teqoin-mainnet-contracts, Tranche A, src/libraries (re-audit of updated code)
Contract Address	Not deployed (undeployed mainnet candidate)

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Risk Classification	2
Audit Summary	3
Audit Scope	4
Audit Method	4
Automated Vulnerability Check	4
Manual Code Review	4
Used tools	4
Table of Contents	5
• Two Different Ways Of Building Proof Trees Are Used Side By Side	6
Code Snippet	6
Recommendation	6
• Some Proof-Checking Machinery Only Matters Once The Fraud System Is Finished	7
Code Snippet	7
Recommendation	7
• The Familiar Never-Reached Safety Checks Appear Here Too	8
Code Snippet	8
Recommendation	8
• A Version Constant Is Named One But Deliberately Holds Zero	9
Code Snippet	9
Recommendation	9
• The System Is Upgradeable, So Governance Ultimately Has Reach Over The Funds	11
Code Snippet	11
Recommendation	11
• Nothing On-Chain Guarantees Every Module Was Built From The Same Core Version	13
Code Snippet	13
Recommendation	13
• The Batch Summary Is The One Major Encoding Without A Cross-Checked Expected Value	15
Code Snippet	15
Recommendation	16
• Correctness Depends On Off-Chain Software Matching These Encodings Exactly	17
Code Snippet	17
Recommendation	18
Disclaimer	19

● Two Different Ways Of Building Proof Trees Are Used Side By Side

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
LIB-I1	Status: Resolved. Both proof-tree recipes are unchanged, which is correct, since each was safe on its own and the two are properly separated so one kind of item can never be mistaken for another. The only recommendation was to make sure both recipes, not just one, are covered by the shared test set used to confirm the separate off-chain software produces identical results. That has been done: there is now a known-answer test for each, both checked against fixed expected values described as matching the other implementation.

Code Snippet

```
// test/LibEncoding.t.sol : both recipes now covered
test_GeneralMerkleGoldenMatrixMatchesRust()      // structured recipe
test_ExecutionTraceGoldenRootMatchesRustV1()     // packed recipe
```

Recommendation

No further action needed. Keep both tests in the routine test run so neither recipe can drift without being noticed.

● Some Proof-Checking Machinery Only Matters Once The Fraud System Is Finished

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
LIB-I2	Status: Acknowledged, and the situation is unchanged. The machinery for checking individual execution steps and individual transactions is untouched, and this update adds more of the same kind: a way to state the canonical claim about what a forced call produced, and a reference encoder for the transaction envelope those calls become. All of it is correct and well separated, and all of it becomes decisive only once the fraud-detection system that would rely on it is finished. Until then, correctness still leans on the trusted operator and dispute path, exactly as noted in the facets and L2 reviews.

Code Snippet

```
// present and correct, but not yet the deciding mechanism:
LibExecutionTrace.verify(...)      // unchanged
LibTransaction.verifyCalldata(...) // unchanged
LibForcedTransaction.resultHash(...) // new in this update
```

Recommendation

Keep this machinery covered by tests so it is ready when the fraud-detection system is completed, and keep the current trust assumption clear in the public documentation for this phase.

● The Familiar Never-Reached Safety Checks Appear Here Too

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
LIB-I3	Status: Acknowledged, kept deliberately. The defensive checks against impossible values are still present here. The original note asked the team to settle on one consistent approach across the project rather than to make a specific change, and they have: the genuinely misleading checks were removed elsewhere, and these harmless ones were kept uniformly. That is a coherent and defensible position.

Code Snippet

```
// e.g. LibDeposit.sol : still present
if (message.sourceChainId == 0 || message.destinationChainId == 0)
    revert ZeroChainId();
```

Recommendation

No action needed. The approach is now consistent across the project.

● A Version Constant Is Named One But Deliberately Holds Zero

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
LIB-I4	Status: New in this review. The new summary used to prove withdrawals is built to match the widely used standard exactly, and that standard requires its first field to be zero. The constant holding that zero is named as though it were version one. The value is correct and the comment explains the intent, so nothing is wrong today. The reason it is worth recording is that the name invites a future reader to treat the zero as a mistake and correct it. Changing it to one would silently alter every withdrawal summary the system produces and break compatibility with the standard tooling this format exists to match, and the failure would appear as proofs simply not matching rather than as an obvious error.

Code Snippet

```
// LibBatchCommitment.sol
bytes32 internal constant OUTPUT_ROOT_VERSION_V1 = bytes32(0);
// named V1, holds 0, and 0 is what the external standard requires.

return keccak256(abi.encode(OUTPUT_ROOT_VERSION_V1, stateRoot,
    messagePasserStorageRoot, latestBlockHash));
```

Recommendation

Rename the constant so its name matches its value, for example to reflect that it is version zero of the external format, and add a short comment stating that the value is fixed by an outside standard and must not be changed. This costs nothing now and removes a trap that would be expensive to diagnose later.

● The System Is Upgradeable, So Governance Ultimately Has Reach Over The Funds

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
LIB-L1	Status: Acknowledged, inherent to the design. The upgrade machinery is unchanged, and this remains a property of any upgradeable bridge rather than a flaw in this code. The protection is not that the power is absent but that it can only be used slowly and visibly. That protection is measurably stronger after this update. A change in the upgrade path now prevents an upgrade from quietly repointing the system at different settings, which had been the one way the enforced delay could have been separated from the value it is measured against. The delay is therefore harder to circumvent than it was at the original review.

Code Snippet

```
// LibDiamond.sol : unchanged
// 'The diamond owner can replace any facet and therefore control all
// bridged funds. The mitigation is entirely external.'

// strengthened by the upgrade-path change:
// DiamondCutFacet now reverts if an upgrade changes the settings pointer.
```

Recommendation

Keep the upgrade controller behind the delayed process and a multi-signature setup at all times, and publish clearly who holds those keys and what the delay is, so users can see the protection for themselves. Reducing this power over time, by completing the fraud-detection system, remains the real long-term path.

● Nothing On-Chain Guarantees Every Module Was Built From The Same Core Version

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
LIB-L2	Status: Still open, unchanged. The shared kernel code is untouched in this update, and so is the situation it described. Every module must have been built against the same version of that kernel for behaviour to be consistent, and nothing visible after deployment can confirm it, because a module built against a slightly different version would look identical from the outside. The version marker intended for the build-and-release process to check is still present and still unused by anything on the chain. This matters slightly more after this update than before, because several modules changed while others did not, so a partial or mismatched release is a more realistic mistake than it was when everything was built at once.

Code Snippet

```
// LibDiamond.sol : unchanged
bytes32 internal constant LIB_VERSION =
    keccak256("TEQOIN_LIB_DIAMOND_V3");
// 'CI asserts all facets report the same value, and the deployment
// rehearsal asserts the on-chain value matches the release tag.'
```

Recommendation

Make the version-marker check a required, blocking step in the release process, and re-run it against the actual deployed addresses during the launch rehearsal, exactly as

the code anticipates. Given that this update changed some modules and not others, confirm every deployed module reports the same value before launch.

● The Batch Summary Is The One Major Encoding Without A Cross-Checked Expected Value

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
LIB-L3	Status: New in this review. The shared test set that confirms the separate off-chain software produces identical results is a real strength of this update, and it now covers most encodings by comparing against a fixed expected value that fails the test run if anything drifts. The batch summary is the exception. Its tests confirm that changing any field changes the result, which is valuable but only proves the encoding is sensitive to its inputs, not that it agrees with the other implementation. The one place the batch summary is compared across implementations simply prints the value for a person to check by hand. This matters because of which encoding it is. The batch summary is the top-level commitment that every batch is identified by, and it is also the encoding that changed most in this update: the format version was raised, four fields were added, and the field order shifted. It is simultaneously the most important encoding to get right and the one most likely to have drifted.

Code Snippet

```
// pinned to a fixed expected value (fails automatically on drift):  
//   Merkle, withdrawals, deposits, forced source, forced result,  
//   token pairs, execution trace, Ll-origin envelope  
  
// batch summary: only self-consistency plus a manual printout  
bytes32 expected = LibBatchCommitment.hash(baseline);  
assertTrue(expected != LibBatchCommitment.hash(changed)); // sensitivity only  
console2.logBytes32(LibBatchCommitment.hash(_header())); // manual check
```

Recommendation

Add a fixed expected value for the batch summary in the same style as the other encodings, so a disagreement between the two implementations fails the test run instead of relying on somebody comparing printed output. Do the same for the genesis marker, since it was renumbered in this update too. This is a small amount of work and closes the last gap in an otherwise thorough set.

● Correctness Depends On Off-Chain Software Matching These Encodings Exactly

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
LIB-M1	Status: Substantially improved, but still a launch condition. This has moved a long way. At the original review the code repeatedly stated that a matching implementation and a shared set of known-answer tests were required before deployment, with no sign that either existed. Both now do, on this side. There is a test set covering the proof trees, withdrawals, deposits, token addresses, transactions, the execution record, and the forced-call encodings, each compared against a fixed expected value described as coming from the other implementation. A canonical reference encoder for the transaction envelope has also been added; it is used only by the tests, which is the correct role for it, and it is checked against a real expected result. What keeps this open is that only one side of the comparison can be seen from here. These tests assert against values stated to match the other implementation, but whether that implementation actually runs the same set, and whether the two are locked to matching versions, cannot be confirmed from this code alone. The consequences if they disagree are unchanged: legitimate withdrawals could fail to be accepted and funds could sit stranded. The reassuring part also stands, which is that a mismatch makes things stop rather than allowing anything to be forged.

Code Snippet

```
// now present on this side, one example of several:
```

```
assertEq(  
    encoded,  
    hex"7ef857a0797f66ef89923d21230185b0c42684ef99550577193e727fd0..."  
);  
assertEq(transactionHash, 0xc0123db6366e0f7de791b36ee3a47db549e909...);  
  
// what cannot be confirmed from this repository:  
//   that the other implementation runs the same set in its own tests,  
//   and that the two are pinned to matching versions.
```

Recommendation

Treat the remaining half as launch-blocking. Confirm the other implementation runs the same shared test set in its own routine testing, so a disagreement fails on both sides rather than only being recorded on one. Pin the two to matching versions and re-run the comparison whenever either changes. Close the one encoding still missing a fixed expected value, noted separately in this report. Once those are done and evidenced, this finding can be closed.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

Advanced Manual **Smart Contract Audit**

August 10, 2026

✂ x.com/coinconsultauids

📌 t.me/coinconsult_tg

Audit requested by

TeQoin

Tranche A, src/l2, re-audit

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	4	1	3	0
● Low-Risk	2	1	0	1
● Medium-Risk	1	0	0	1
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

TeQoin

Website	teqoin.io
Blockchain	Ethereum (L1) / TeQoin (L2)
Source Code	teqoin-mainnet-contracts, Tranche A, src/l2 (re-audit of updated code)
Contract Address	Not deployed (undeployed mainnet candidate)

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Risk Classification	2
Audit Summary	3
Audit Scope	4
Audit Method	4
Automated Vulnerability Check	4
Manual Code Review	4
Used tools	4
Table of Contents	5
• A Duplicate-Message Guard That Can Never Actually Trigger	6
Code Snippet	6
Recommendation	6
• Repeated Safety Checks That Cannot Be Reached	7
Code Snippet	7
Recommendation	7
• Deposited ETH Is Permanently Locked On The L2 Side, By Design	8
Code Snippet	8
Recommendation	8
• A Deferred Deposit Looks Like A Completed One Unless The Extra Event Is Watched	9
Code Snippet	9
Recommendation	9
• The L2 Side Trusts The Deposit Processor To Reflect Only Real L1 Deposits	11
Code Snippet	11
Recommendation	11
• A Withdrawal Can Be Started To An L1 Address That Cannot Receive The Funds	13
Code Snippet	13
Recommendation	13
• An ETH Deposit To A Recipient That Rejects Payments Cannot Complete And Has No Refund	15
Code Snippet	15
Recommendation	16
Disclaimer	17

● A Duplicate-Message Guard That Can Never Actually Trigger

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
L2-I1	Status: Acknowledged, unchanged. The contract that records outgoing withdrawal messages is untouched in this update, so the duplicate-message guard that can never fire is still present. Each message carries a counter that goes up every time, which already guarantees two messages can never come out identical. Keeping the guard as belt-and-braces was one of the two acceptable options, and it is consistent with how similar defensive checks have been kept elsewhere in the project.

Code Snippet

```
// L2ToL1MessagePasser.sol : unchanged in this update
uint256 nonce = messageNonce++; // always unique
if (sentMessages[withdrawalId]) revert DuplicateMessage(withdrawalId);
// can never already be true
```

Recommendation

No action needed. A one-line comment noting that the counter already guarantees uniqueness would save a future reader the same analysis.

● Repeated Safety Checks That Cannot Be Reached

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
L2-I2	Status: Acknowledged, kept deliberately. The checks against a network identifier of zero, and the check that a freshly created token landed at its predicted address, are still present. The original note asked the team to settle on one consistent approach rather than to make a specific change. They have done exactly that across the project: the genuinely misleading checks were removed, and these harmless defensive ones were kept uniformly. That is a coherent position.

Code Snippet

```
// L2Bridge.sol : still present
if (l1ChainId_ == 0 || block.chainid == 0) revert ZeroChainId();

// L2TokenFactory.sol : unchanged in this update
if (l2Token != expected) revert DeploymentAddressMismatch(expected, l2Token);
```

Recommendation

No action needed. The approach is now consistent across the project.

● Deposited ETH Is Permanently Locked On The L2 Side, By Design

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
L2-I3	Status: Acknowledged, and the accounting has improved. The design is unchanged and remains intentional: ETH locked on this side is deliberately taken out of circulation, the matching payout happens on the main chain instead, and there is purposely no way to sweep it out because a sweep would create a privileged path to move funds. The accounting around it has been made more accurate in this update. The new deferred-credit mechanism means some ETH can now be owed to a specific recipient rather than only to withdrawals, and both the solvency check and the spare-balance figure were correctly updated to count it. That is the right change and it keeps the solvency figure meaningful.

Code Snippet

```
// L2Bridge.sol : liabilities now include deferred deposit credit
function isNativeSolvent() external view returns (bool) {
    return address(this).balance
        >= nativeWithdrawalLiability + totalPendingNativeDepositCredit;
}
```

Recommendation

No change needed. Keep the solvency helper in ongoing monitoring so any unexpected shortfall would be noticed.

● A Deferred Deposit Looks Like A Completed One Unless The Extra Event Is Watched

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
L2-I4	Status: New in this review, arising from the improved deposit handling. The fix for the stuck-deposit problem is a good one: when ETH cannot be handed straight to the recipient, the deposit still completes and the amount is set aside for that recipient to collect later. A side effect is that the deposit now reports success in exactly the same way whether or not the recipient actually received anything. The usual completion event is recorded either way, and the only signal that the money is sitting waiting is a second, separate event. The risk is not to the funds, which are safely held and claimable. It is that a user or a monitoring dashboard watching only the completion event will believe the ETH arrived when it did not, and the amount could sit unclaimed indefinitely because nobody knows to collect it.

Code Snippet

```
// L2Bridge.sol : both paths end with the same completion event
if (!ok) {
    pendingNativeDepositCredit[message.to] += message.amount;
    emit NativeDepositCreditDeferred(depositId, message.to, message.amount);
}
...
emit DepositProcessed(...); // recorded either way
```

Recommendation

Have the front end and any monitoring watch for the deferred-credit event alongside the completion event, and show affected users a clear prompt to collect their funds. A

simple lookup already exists for checking whether an address has an unclaimed amount, so surfacing it is straightforward. Consider alerting if a deferred amount stays unclaimed for a long period.

● The L2 Side Trusts The Deposit Processor To Reflect Only Real L1 Deposits

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
L2-L1	Status: Improved, but the underlying assumption remains. One real improvement has been made. The special account allowed to credit deposits is no longer supplied when the contract is created; it is now a fixed, well-known system address written into the code. That removes any possibility of it being set to the wrong value at deployment, and it removes an operator key from the picture entirely. What remains is the assumption that was always the real dependency. The contracts on this side still do not independently verify that a deposit really happened on the main chain, because that verification belongs to the chain's derivation and fraud-detection system, which is incomplete in this phase. Until that is finished, correctness of L2 crediting rests on the trusted path rather than on automatic checking.

Code Snippet

```
// L2Bridge.sol : no longer a deployment parameter
address public constant DEPOSIT_PROCESSOR =
    address(uint160(0xDeaDDeaDDeAdDeAdDEAdDEaddeAddeAdDEAd0001));
// 'deliberately not a constructor parameter or operator key.'
```

Recommendation

State this trust assumption plainly in the public documentation for this phase, so users understand that L2 crediting depends on the trusted path until the fraud-detection system is finished. Completing that system is the real resolution and is already the

project's stated direction. Confirm as part of launch checks that the fixed address written into the code is the one this chain actually uses.

● A Withdrawal Can Be Started To An L1 Address That Cannot Receive The Funds

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
L2-L2	Status: Resolved, by the fix on the main-chain side. The original concern was that a user starting a withdrawal chose a destination that was never checked for whether it could actually receive the funds, and because the tokens were burned immediately, a poor choice could leave the value stranded with no way to change the destination. The dead end has been removed exactly where recommended, on the main chain rather than here. A withdrawal that has been proven but not yet paid out can now be pointed at a different destination, with the amount and the underlying proof untouched. Only the destination named in the proof can authorise the change, which closes the recovery gap without letting anyone take back a payment. This side of the bridge is unchanged and does not need to change.

Code Snippet

```
// L2Bridge.sol : unchanged, destination still chosen freely
IL2StandardToken(l2Token).bridgeBurn(msg.sender, amount);

// the recovery route now exists on the main chain:
// BridgeFacet.redirectProvenWithdrawal(message, newRecipient)
// callable only by the address named in the proof.
```

Recommendation

No further action needed in these contracts. Keep warning users to check their destination carefully, since the remaining stuck case is a destination contract that cannot

make any call at all, and make sure the redirect route is documented where a user in that position would find it.

● An ETH Deposit To A Recipient That Rejects Payments Cannot Complete And Has No Refund

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
L2-M1	Status: Resolved. Previously an ETH deposit was delivered by sending the funds straight to the recipient, and if that recipient refused the payment the whole deposit could not be completed. Because there is no way to cancel or refund a deposit on the main chain, the locked funds were then stuck with no route back. The fix is the one recommended, and it is well built. The delivery attempt is still made, but it is now given a limited amount of processing so a hostile recipient cannot exhaust the transaction, and if it fails the deposit no longer fails with it. The amount is instead set aside for that recipient and the deposit completes normally. The recipient can then collect it at any time, and can send it to a different address of their choosing, which also covers the case where their own address cannot hold funds. If that collection attempt fails, the record is left intact so it can be tried again. The solvency accounting was correctly updated to include amounts held this way.

Code Snippet

```
// L2Bridge.sol : delivery is now best-effort with a safe fallback
(bool ok,) = message.to.call{ value: message.amount,
                             gas: NATIVE_DEPOSIT_PUSH_GAS }("");
if (!ok) {
    pendingNativeDepositCredit[message.to] += message.amount;
    totalPendingNativeDepositCredit += message.amount;
    emit NativeDepositCreditDeferred(depositId, message.to, message.amount);
}

// the recipient can collect later, to any destination they choose:
function claimNativeDepositCredit(address payable destination) external
```

```
nonReentrant returns (uint256 amount)
```

Recommendation

No further action needed to clear this finding. See the separate note in this report about making sure users are actually told when their deposit was set aside rather than delivered, since the funds are only safe in practice if somebody knows to collect them.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

Advanced Manual **Smart Contract Audit**

August 10, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

TeQoin

Tranche A, src/governance, re-audit

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	4	1	1	2
● Low-Risk	3	1	2	0
● Medium-Risk	1	0	0	1
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

TeQoin

Website	teqoin.io
Blockchain	Ethereum (L1) / TeQoin (L2)
Source Code	teqoin-mainnet-contracts, Tranche A, src/governance (re-audit of updated code)
Contract Address	Not deployed (undeployed mainnet candidate)

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Risk Classification	2
Audit Summary	3
Audit Scope	4
Audit Method	4
Automated Vulnerability Check	4
Manual Code Review	4
Used tools	4
Table of Contents	5
• Executors Are Not Checked The Way Proposers And Guardians Are	6
Code Snippet	6
Recommendation	6
• The Deployment Wiring Check Leaves Out One Address	8
Code Snippet	8
Recommendation	8
• The Announced Summary Of An Upgrade Is A Note, Not A Separate Lock	9
Code Snippet	9
Recommendation	9
• The Deployment Now Pins Three Trusted Starting Values Instead Of One	10
Code Snippet	10
Recommendation	11
• Lowering One Safety Timer Quietly Loosens Several Protections At Once	12
Code Snippet	12
Recommendation	12
• All Governance Depends On One Settings Contract Always Being Readable	14
Code Snippet	14
Recommendation	14
• The Data-Storage Requirement Can Only Be Raised, Never Lowered	15
Code Snippet	15
Recommendation	15
• The Timelock And The Rest Of The System Can End Up Reading Different Settings	16
Code Snippet	16
Recommendation	17
Disclaimer	18

● Executors Are Not Checked The Way Proposers And Guardians Are

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
GOV-I1	Status: Resolved. The original note asked the team to confirm that leaving the accounts which carry out approved actions unchecked was deliberate rather than an oversight. It was deliberate, and the intent is now written into the code rather than left implied. The arrangement where anyone may carry out an already-approved and fully-delayed action is now recognised explicitly, it is exposed as a public flag so anyone can see which mode a deployment is in, and a mixed set that would leave the intent ambiguous is now rejected outright. Accounts in this role still are not required to be contracts, and that is correct. Unlike the accounts that propose or cancel actions, these accounts cannot schedule or change anything, so an ordinary wallet acting as an automated helper is a legitimate choice.

Code Snippet

```
// TeQoinTimelock.sol : intent now explicit and enforced
bool public immutable OPEN_EXECUTION;

bool openExecution = executors.length == 1 && executors[0] == address(0);
// a mixed set would be ambiguous, so it is refused:
if (executors[i] == address(0) && !openExecution)
    revert InvalidOpenExecutorSet();
```

Recommendation

No further action needed. Confirm at deployment which mode is intended, since the flag is fixed permanently when the timelock is created.

● The Deployment Wiring Check Leaves Out One Address

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
GOV-I2	Status: Resolved. The deployment check now covers the factory that creates matching tokens on the other chain, which was the one key address it previously omitted. It has been added at every level: the registry now exposes it, it is a required deployment value, a blank value is refused, and it is cross-checked against the registry in the same way as the neighbouring addresses. This is the same change that clears the corresponding finding in the token-registry review, and this is where the fix belonged.

Code Snippet

```
// DiamondInit.sol : updated
function L2_TOKEN_FACTORY() external view returns (address); // now exposed
if (a.l2TokenFactory == address(0)) revert ZeroAddress("l2TokenFactory");

_requireAddress("tokenPairRegistry.l2TokenFactory",
    a.l2TokenFactory, registry.L2_TOKEN_FACTORY());
```

Recommendation

No further action needed.

● The Announced Summary Of An Upgrade Is A Note, Not A Separate Lock

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
GOV-I3	Status: Acknowledged, unchanged by design. The upgrade announcement system is unchanged in this update, which is appropriate because no change was recommended. The plain-language summary attached to an announced upgrade remains a human-readable label rather than something the contract separately enforces, and that is fine, because the announcement already locks in the exact upgrade code and the exact instructions it will run. Those together fully determine what an upgrade does.

Code Snippet

```
// UpgradeRegistry.sol : unchanged in this update
// the announcement binds the exact change, the exact upgrade code
// (by its code fingerprint) and the exact instructions it runs.
// the summary sits alongside as a label.
```

Recommendation

No action needed. Keep reviewers instructed to read the bound upgrade code and instructions rather than relying on the summary alone.

● The Deployment Now Pins Three Trusted Starting Values Instead Of One

Informational - Does not compromise the functionality of the contract in any way

Error Code	Description
GOV-I4	Status: New in this review, arising from the updated batch format. Batches now commit to a combined summary built from three values rather than one: the state of the chain, the record used to prove withdrawals, and the identifying fingerprint of the final block. Each of these is set for the very first batch by the deployment itself, and each is simply trusted to be correct. The contract refuses blank values, which is a useful guard against an obvious mistake, but nothing can check that a non-blank value is the right one. This is not a weakness in the design, and committing to all three is the stronger arrangement. The point is that the launch checklist has grown: a wrong starting value for the withdrawal record in particular would affect how withdrawals are proven against the very first batch, so all three now need to be verified against the real starting state of the other chain rather than just the one.

Code Snippet

```
// DiamondInit.sol : three trusted starting values, blank-checked only
if (a.genesisStateRoot == bytes32(0)) revert ZeroGenesisStateRoot();
if (a.genesisL2BlockHash == bytes32(0)) revert ZeroGenesisL2BlockHash();
if (a.genesisMessagePasserStorageRoot == bytes32(0)) { ... }

// all three feed the combined summary used from then on:
keccak256(abi.encode(OUTPUT_ROOT_VERSION_V1, stateRoot,
    messagePasserStorageRoot, latestBlockHash))
```

Recommendation

Add all three starting values to the launch verification checklist and confirm each against the real starting state of the other chain before anyone relies on the bridge, rather than confirming only the chain state as before. This pairs with the existing recommendation to publish a fingerprint of the deployed setup so the whole configuration can be checked in one step.

● Lowering One Safety Timer Quietly Loosens Several Protections At Once

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
GOV-L1	Status: Improved, but still open. The central safety timer still feeds several other protections at once, and lowering it still relaxes all of them together. Nothing in this update prevents that, and it remains possible to reduce the timer without any transaction touching the vaults. What has improved is that the most surprising consequence is no longer silent. The vaults now refuse a limit window longer than the current timer at the moment it is set, and they expose a monitoring view that reports explicitly when the window has ended up longer than the timer. So the situation this finding warned about can now be seen immediately rather than discovered during an incident. The coupling itself, and the need to review the whole system when the timer changes, is unchanged.

Code Snippet

```
// the central timer still feeds, at least:  
// - the minimum delay on governance actions (TeQoinTimelock)  
// - the withdrawal speed limit in both vaults  
//  
// newly detectable, but not prevented:  
//   RateLimitStatus.WindowExceedsChallengePeriod
```

Recommendation

Treat any reduction of this timer as a coordinated review across the whole system rather than a single setting change, and check the effect on both vaults and on the governance

delay each time. Wire the vaults' new monitoring view into alerting so that if the two ever do drift out of step it is caught immediately.

● All Governance Depends On One Settings Contract Always Being Readable

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
GOV-L2	Status: Acknowledged, unchanged by design. The timelock still reads the safety timer from the settings contract before scheduling or carrying out any action, so a failure to read it would stop upgrades and parameter changes, though cancelling would still work. This is a deliberate fail-safe: it refuses to act rather than act on missing information, and that is the right direction for a governance control. The likelihood remains very low, because the settings contract is fixed at deployment, cannot be swapped, and the value it returns is a simple stored number. One point has strengthened since the original review. A separate change now prevents an upgrade from repointing the rest of the system at a different settings contract, so the assumption that everything reads the same one is now enforced rather than merely expected.

Code Snippet

```
// TeQoinTimelock.sol : unchanged, deliberate
// getMinDelay() reads SYSTEM_CONFIG.challengePeriod() with no fallback;
// scheduling and executing both depend on it. Cancelling does not.
```

Recommendation

No code change needed. Keep the settings contract as simple as it is now and include it in ongoing monitoring.

● The Data-Storage Requirement Can Only Be Raised, Never Lowered

Low-Risk - Won't cause any problems, but can be adjusted for improvement

Error Code	Description
GOV-L3	Status: Acknowledged, unchanged by design. The requirement for where batch data must be stored can still only ever be tightened, never relaxed. This remains a sound security stance, since weakening how batch data is made available would be dangerous, and the update leaves it as it was. The trade-off is unchanged: if a chosen storage method later becomes unworkable, governance cannot step back to an earlier one and would have to work around it another way. There is no impact in normal operation.

Code Snippet

```
// SystemConfig.sol : unchanged
if (newMode <= latest.mode) revert DaModeNotIncreasing(latest.mode, newMode);
// the requirement can move up but never back down.
```

Recommendation

No action needed beyond confirming this is the intended long-term stance and recording it as an accepted trade-off.

● The Timelock And The Rest Of The System Can End Up Reading Different Settings

Medium-Risk - Will likely cause problems and it is recommended to adjust

Error Code	Description
GOV-M1	Status: Resolved. This was the main finding in the original governance review. The timelock is permanently locked to one settings contract, while the rest of the bridge reached its settings through a second reference that an upgrade could have changed. If the two had ever drifted apart, the timelock would have kept enforcing its delay from one source while the bridge ran on another, breaking the promise that governance changes come with a full period of advance warning. The fix is a direct one and lands in the upgrade path itself. The upgrade routine now records which settings contract the system is using before an upgrade runs, and refuses the whole upgrade if the instructions changed it. The two references can therefore no longer be separated by an ordinary upgrade. The code explains the reasoning in the same terms this finding raised, and a specific error was added so the refusal is unambiguous.

Code Snippet

```
// DiamondCutFacet.sol : the two references can no longer drift apart
address systemConfig = governance.systemConfig;      // captured before
...
address currentSystemConfig = governance.systemConfig;
if (currentSystemConfig != systemConfig)
    revert SystemConfigChanged(systemConfig, currentSystemConfig);
// 'Allowing an initializer to swap only the diamond-side pointer
// would split governance policy.'
```

Recommendation

No further action needed. If the settings contract ever genuinely must be replaced, it now has to be done as an explicit coordinated migration that replaces the timelock at the same time, which is the correct and intended consequence of this change.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.